

Is it possible to restore the files of a specific commit without losing the subsequent commits?

Yes, absolutely! There are several ways to restore files from a specific commit without losing subsequent commits. Here are the most common approaches:

1. Restore specific files using `git checkout`

```
bash

# Restore specific files from a commit
git checkout <commit-hash> -- <file1> <file2>

# Example: restore index.html from commit abc123
git checkout abc123 -- index.html
```

This brings the file(s) from the specified commit into your working directory and stages them. Your commit history remains intact.

2. Using `git restore` (Git 2.23+)

```
bash

# Restore files from a specific commit
git restore --source=<commit-hash> <file1> <file2>

# Example
git restore --source=abc123 index.html
```

This is the modern way to restore files and doesn't stage them automatically.

3. Extract files without modifying working directory

```
bash

# Show file content from a specific commit
git show <commit-hash>:<file-path>

# Save file content to a new location
git show <commit-hash>:<file-path> > restored-file.txt
```

4. Interactive approach with `git log`

```
bash
```

```
# Find the commit you want
git log --oneline

# Then use checkout or restore as above
```

Key points:

- **No commits are lost** - you're only modifying files in your working directory
- **History stays intact** - all subsequent commits remain in the repository
- **You control what happens next** - you can commit the restored files as a new commit, or just use them temporarily
- **Selective restoration** - you can restore just specific files, not the entire project state

After restoring files, you can commit the changes if you want to make the restoration part of your project's history, or just use the restored files without committing.

What is the fastest method?

The **fastest method is** `git checkout <commit-hash> -- <file>`.

Here's why:

```
bash

git checkout abc123 -- file.txt
```

Speed advantages:

- Single command that both retrieves AND stages the file
- No need for a separate `git add` step
- Works in all Git versions (unlike the newer `git restore`)
- Minimal typing and cognitive overhead

Comparison:

- `git restore --source=abc123 file.txt` requires an additional `git add` if you want to stage
- `git show abc123:file.txt > file.txt` requires redirection and doesn't handle binary files well
- Interactive methods involve multiple steps

Pro tip for maximum speed:

```
bash
```

```
# You can use short commit hashes (first 7 characters usually work)
git checkout abc1234 -- file.txt
```

```
# Restore multiple files at once
git checkout abc1234 -- file1.txt file2.txt folder/
```

The only scenario where `git restore` might be faster is if you specifically don't want the files staged, but in most cases where you're restoring files, you probably want to commit them anyway, making `git checkout` the clear winner for speed and efficiency.

What is the fastest method if I don't know the commit hash?

The **fastest method when you don't know the commit hash** is using relative references or search patterns:

1. Relative references (fastest for recent commits)

```
bash
```

```
# From 3 commits ago
git checkout HEAD~3 -- file.txt

# From the previous commit
git checkout HEAD~1 -- file.txt
# or simply
git checkout HEAD^ -- file.txt
```

2. Branch references (fast for branch tips)

```
bash
```

```
# From another branch
git checkout main -- file.txt
git checkout feature-branch -- file.txt
```

3. Quick hash lookup with grep (fast for known commit messages)

```
bash
```

```
# Find and use in one line
git checkout $(git log --oneline | grep "fix bug" | head -1 | cut -d' ' -f1) -- file.txt
```

4. Interactive selection with fzf (fastest for fuzzy searching)

If you have `fzf` installed:

```
bash

# Interactive commit picker
git checkout $(git log --oneline | fzf | cut -d' ' -f1) -- file.txt
```

5. Date-based (fast when you know roughly when)

```
bash

# From yesterday
git checkout 'HEAD@{yesterday}' -- file.txt

# From 2 days ago
git checkout 'HEAD@{2.days.ago}' -- file.txt
```

6. Quick commit browsing

```
bash

# See last 5 commits with one-line format
git log --oneline -5

# Then copy-paste the hash
git checkout a1b2c3d -- file.txt
```

Fastest overall: `HEAD~N` references if you know roughly how many commits back, otherwise `git log --oneline -10` to quickly spot and copy the hash you need.